

Whisper STT Implementation Lab — Report

Audio: MS072.mp3 (2:13, oral history interview) | Model: whisper-1

1. Prompted vs. Unprompted Transcription

Three transcription runs were compared on the same audio file to evaluate the effect of the *prompt* parameter:

Version	Word Count	Notes
Unprompted (baseline)	323	Full content, no guidance given
Descriptive prompt	269 (-17%)	Dropped the opening ~4 sentences of real audio content
Keyword-style prompt	323	Full content retained; improved dialect spelling (“feller” vs “fellow”)

Finding: A descriptive, summary-style prompt (“This is an oral history interview about a family's relocation...”) caused Whisper to omit a full opening passage of real speech — the model appears to over-anchor on the prompt's framing and treat the audio as starting mid-conversation. A short, keyword-only prompt (“Miss Davis, Gainesville, chicken yard, chickens”) preserved all content while still nudging dialect spelling toward more natural forms. This suggests prompting is not purely additive for Whisper: descriptive prompts risk content loss, while terse keyword prompts are safer and still improve accuracy on names and domain vocabulary.

2. Benefits of Chunking for Long Audio

The Whisper API enforces a 25MB file size limit per request. MS072.mp3 (2.03MB, 133s) was well under this limit, so a single chunk was sufficient for the real transcription run. To validate the chunking logic itself, a second pass was run with a 30-second chunk length, correctly producing 5 chunks (0–30s, 30–60s, 60–90s, 90–120s, 120–133s), confirming the split-and-offset logic works correctly regardless of file size. At scale (e.g. 2+ hour meeting recordings), chunking is essential: it keeps individual requests under the size limit, allows partial results to be processed and reviewed incrementally, and isolates failures to a single chunk rather than losing an entire multi-hour transcription job if one request fails.

3. Challenges Faced

- **Environment mismatch:** Packages installed via a plain PowerShell terminal landed in the base Anaconda environment, not the conda environment (lab) actually used by the Jupyter kernel, causing repeated “ModuleNotFoundError” despite a successful-looking pip install . . . , which guarantees the package lands in the kernel's own environment.
- **Python 3.13 / pydub incompatibility:** Python 3.13 removed the built-in audioop module, which pydub depends on internally, causing ModuleNotFoundError: No module named 'audioop'. Resolved by installing the audioop-lts backport package.
- **Timestamp offsetting:** Whisper resets segment timestamps to start at 0 for every chunk sent independently, so each chunk's segment times had to be manually offset by the chunk's start time in the original file to produce a continuous, accurate timeline.

4. Recommendations for Improving Accuracy

- Prefer short, keyword-style prompts (names, technical terms, jargon) over descriptive summary prompts — the latter risks the model dropping real content near the start of the audio.
- Check file size before assuming chunking is required; only split when approaching the 25MB API limit, but still test the chunking logic on a shortened chunk length to confirm it works before relying on it for a real long recording.
- Always verify pip installs target the same Python environment the notebook kernel is using (sys.executable), especially in conda setups with multiple environments.
- Use response_format="verbose_json" with segment-level granularity for any use case needing navigation or search (e.g. subtitle generation, meeting minute lookup).